הרצאה 1 – שפות תכנות 2027

מרצה: הדר פרנקל

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

מנהלות והקדמה

- המטלות לא חובה: הן מגן. אם מגישים – להגיש הכל. 4 מטלות. המטלות בית בזוגות.
- מבחן חובה: 70%. חובת מעבר.
- מבנה המבחן יישאר זהה כבשנים עברו.
- סילבוס זהה סה"כ לשנים קודמות. מצגות שונות. סדר שונה.

למה שפות תכנות? קצת מוטיבציה. חלק משפות התכנות שנראה ממש מתבססות על חוקים מתמטיים שלמדנו בשנים עברו. נלמד כיצד לכתוב דרישות פורמליות משפת התכנות שאנו משתמשים בה, ונלמד סמנטיקה וכיצד לבדוק את נכונות התוכניות שאנחנו כותבים. כמו כן, נלמד פרדיגמות תכנות נוספות ונרחיב על מודלים חישוביים – נלמד מודל חישובי מתמטי ששקול למכונת טיורינג.

מה זה תכנות? באופן כללי – מדבר על נתונים ועל פעולות שאנחנו מבצעים על אותם נתונים. ישנם פרדיגמות תכנות שונות:

- **תכנות לוגי** – תכנות שבו מבצעים מניפולציות על הנתונים. מגדירים עובדות וחוקים על הנתונים.
- **תכנות אימפרטיבי** – שפות תכנות כמו java, python וכו'. רצף של פקודות שאומרות לנו כיצד לתמרן ולשנות את הנתונים בתוכנית.
- **תכנות פונקציונלי** – תכנות שלא מדבר על הנתונים וסה"כ מגדיר פונקציות.

זוג הקצוות נקרא **תכנות הצהרתי** – הוא אומר מה רוצים לעשות, ולא כיצד עושים זאת. למשל prolog אומרת מה רוצים לעשות: המימוש – אומר איך.

Prolog

פרולוג היא שפה שמצהירה לנו **עובדות וחוקים** על הדטה. באמצעות ההצהרות הללו אנחנו מתכנתים. פרולוג היא שפה בה אנחנו משתמשים בתכנות הצהרתי. זהו תכנות לוגי.

לפני שנתחיל בסינטקס של פרולוג עלינו להבין את אבני הבניין הבסיסיים.

- בפרולוג הכל בנוי על terms: התוכניות עצמן והנתונים שעושים עליהם מניפולציה. ישנם 3 סוגים של ביטויים:

1. **<u>קבועים:</u>** אטומים, מחרוזות. למשל קבועים מסוג int, מספרים ממשיים, אטומים: אטומים מתחילים **באות קטנה**, לא באות גדולה! אחרת זו הצהרת משתנה. אטום למעשה הוא מחרוזת: זהו לא משתנה אלא קבוע שלצורך העניין נקרא לו fred.
2. **<u>משתנים:</u>** מתחילים תמיד **באות גדולה** או ב_. לרוב אנחנו לא ניתן למשתנים שם שמתחיל ב_, אלו לרוב משתנים פנימיים שפרולוג עושה את החישובים איתם. משתנה עם _ הוא משתנה אנונימי שלא אכפת לנו מהערך שלו לרוב – נרחיב על זה בהמשך הקורס.
3. **<u>Compound terms:</u>** רשימות ומבני נתונים יותר מתקדמים. נקרא לביטויים אלו מבנים. ניתן לחשוב על זה בתור יחס מעל terms – אוסף הביטויים.
   למשל x(y,z) הוא – x הוא יחס על y,z. דוגמה נוספת: (1,2)+. + הוא יחס על 1 ו2. כמובן שניתן לראות דברים כמו ((1,2)x(Y,x וכו' – הקריאה יכולה להיות רקורסיבית.
   ליחס כזה נקרא functor: יחס מעל שני משתנים, שלרוב בראש יכול לקבל משמעות אך מבחינת פרולוג כמובן אין לו כל משמעות. לfunctor יש מס' ארגומנטים – וזה נקרא הדרגה של הfunctor. למשל הדרגה של f(a,b,c) היא 3.

- ביטוי סה"כ יכול להיות קבוע, משתנה או מבנה מורכב.

כעת, ניתן להתחיל לדבר על פרולוג עצמה: ובעיקר – כיצד תוכניות נראות.

- פרולוג שומרת אוסף של עובדות על מסד הנתונים שלנו, ואוסף של חוקים שמהם אפשר להסיק כל מיני דברים. על אוסף העובדות והחוקים ניתן לכתוב שאלות.

עובדות: עובדה נכתבת באופן הבא כמו מצד שמאל. בדוגמה – עובדה מסוג "אבא" שבה kim הוא אבא של holly. כבר על עובדה פשוטה זו ניתן לשאול שאלות.

```
parent (Kim, Holly)
parent (Kim, guy)
```

בפרולוג עצמו – היכן שמופיע סימן השאלה ? ניתן להוסיף שאלות. אנחנו ראשית טוענים את קובץ העובדות באמצעות consult(relations) כאשר relations.pl הוא שם הקובץ של העובדות. נשים לב כי בשלב זה כי רק מה שהכנסנו למסד הנתונים יחזיר TRUE. כל דבר אחר – לא. בהמשך, כשנגדיר חוקים אם יהיה אפשר להסיק מהם דברים שלא כתובים במסד הנתונים אך החוקים כן מגדירים: עליהם יוחזר TRUE. **אלו נקראות שאילתות.**

```
?-parent(Kim, Holly).
True.
?-parent(Kim, Guy).
False.
```

- כל שורת פרולוג חייבת להסתיים בנקודה. אם לא נעשה נקודה, הפרולוג פשוט יחכה.

ניתן לשאול גם שאילתות יותר מורכבות. למשל:

```
?- parent(X, holly).
X = Kim.
```

כאן X הוא משתנה. פרולוג יבדוק אם יש דרך להשלים את המשתנה הזה כך שהעובדה הנ"ל תהיה נכונה. במקרה שלנו – יש לו דרך. לכן יחזיר קים. אם יש כמה תשובות לשאלה: הוא יחזיר את הראשונה לפי סדר מסד הנתונים, אם נעשה ; נוכל לקבל את ההבאים. ברגע שאין עוד אפשרויות – הוא יחזיר false.

- אם יוחזר false מיידית, זה אומר שאין ערך למשתנה X שיהפוך את העובדה הנ"ל לנכונה.

כמו כן, ניתן לבצע שאילתא עם כמה משתנים. למשל:

```
?- parent(X, Y).
X = Kim.
Y = holly.
```

כלומר הוא מחזיר את הזוג שמקיים. אם קיימים עוד, הוא יחזיר עוד בהינתן ;. חשוב להבין שהוא מחזיר את התשובות משמאל לימין <u>לפי הסדר שכתובים במסד הנתונים</u>.

- קשר "וגם": אם נרצה לשאול יותר משאילתא אחת נעשה זאת באמצעות פסיק. למשל:

```
?- parent(X, holly), parent(X,guy)
X = Kim.
```

מה שכאן שאלנו: האם ישנו X שהוא הורה גם של holly וגם של guy – יש כזה: Kim.

<u>**דוגמה נוספת**</u> – האם לקים יש סבא? כלומר – האם לקים יש הורה, ולאותו הורה יש גם הורה כלשהו. נשים לב כי Parent, Grandparent הם משתנים.

```
?- parent(Parent, kim), parent(Grandparent, Parent)
```

<u>**עד כה דיברנו על עובדות. כעת, נדבר על חוקים**</u>. מה אם נרצה לבדוק אם מישהו הוא צאצא של מישהו אחר? זה עבודה קשה לעבוד כמו בדוגמה לעיל. הhead של החוק הוא מה שמופיע בתחילתו משמאל. ה-: הוא למעשה חץ, לכיוון שמאל. אם שלושת התנאים מתקיימים: אז החוק נכון. נתבונן כיצד נראה חוק:

```
Greatgrandparent(GGP,GGC) :-
Parent (GGP,GP),
Parent(GP,P),
Parent(P,GGC)
```

למשל: חוק שבו נרצה להגדיר מתי GGP הוא הסבא רבא של GGC. מתי? אם קיים GP שהוא הבן של GGP, וקיים P שהוא הבן של GP, וגם P הוא ההורה של GGC.

כל חוק לרוב נשתמש בו עם משתנים.

- התוכנית מורכבת מרשימה של clauses "Horn". Clause הוא או עובדה או חוק שצריך להסתיים בנקודה.

כעת, ניתן לבצע שאילתא גם על החוקים שלנו. באופן דומה לחלוטין לאיך שהשתמשנו קודם. למשל:

כאן הפעלנו את הכלל שקודם הגדרנו – האם לholly יש סבא רבא?

```
?- greatgrandparent(X, holly).
```

- הסדר שבו כתבנו את החוקים – משנה. הוא עובד משמאל לימין גם בתוך כל חוק, ומנסה קודם משמאל עד לימין. מדוע זה משנה? בהמשך נראה תוכניות ברקורסיה והסדר ישנה, אחרת נכנס ללולאות אינסופיות. הוא גם עובד מלמעלה ללמטה.

כיצד עובד התהליך? ראשית לתוך הכלל נכנסים הקבועים – במקרה שלנו GGC=holly. ואז מתחילים לחפש במסד הנתונים האם ניתן לספק את החוקים, אחד אחר השני.

- ניתן – אם רוצים: לבקש מפרולוג את הtrace של החישוב, תהליך החישוב עצמו.
- נשים לב כי קשרים סימטריים ולא סימטריים: פרולוג כמובן לא יודע להבחין בזה. למשל: גיא אח של עומר, אבל גם עומר אח של גיא. אם הכנסנו למסד הנתונים רק $b(guy, Omer)$ אז לא ידוע כי גם עומר הוא אח של גיא. יש להגדיר זאת בעצמנו עם חוקים! באופן דומה: שירה היא אמא של דנה, אבל דנה לא אמא של שירה. אם במסד הנתונים במקרה יש לנו $m(dana, Shira)$,$m(Shira, dana)$ אז יש כאן סתירה – אבל פרולוג לא יודע את זה. אנחנו צריכים לאכוף את זה.

ניתן להגדיר חוקים שמתבססים על חוקים קיימים. למשל: במקום להגדיר סבא-רבא כפי שהגדרנו, ניתן להגדיר סבא ובאמצעותו להגדיר סבא-רבא כך:

```
grandparent (GP,GC) :-
Parent(GP,P), parent(P,GC).
greatgrandparent(GGP,GGC) :-
grandparent(GGP,P), parent(P,GGC).
```

- נשים לב כי P בשני המקרים כאן הוא לא זהה. הscope שלו הוא עד הנקודה! כלומר הP של grandparent הוא לא אותו P שמופיע בהגדרת greatgrandparent.
- יש לשים לב שחייבים להגדיר את סבא מעל סבא-רבא, כי הוא משתמש בו.

<u>חוקים רקורסיביים:</u>

בפייתון או בג'אווה יש לנו לולאות – כאן אין. מבחינת הגדרה, נהיה חייבים לעבוד כמעט תמיד עם רקורסיה. למשל: כיצד נגדיר צאצא?

```
Ancestor (X,Y):- parent(X,Y).
Ancestor (X,Y) :-
Parent (Z,Y), ancestor(X,Z).
```

כלומר הגדרנו פעמיים – פעם אחת למקרה ישיר. אם X הוא ההורה של Y. אחרת, אזי מההגדרה הרקורסיבית: קיים Z שהוא ההורה של Y, וכן Z צאצא של X. **חשיבה רקורסיבית, תמיד כמעט תעבוד כאן בצורה זו באופן פשוט מאוד.**

למעשה כיצד חושב פרולוג? הוא מנסה להוכיח שY הוא צאצא של X. אז הוא הולך למקרה הבסיס, אם לא הוא הולך ל"צעד" ומפרק את הבעיה לבעיה רקורסיבית ע"י מציאת Z שיאפשר להתקדם ברקורסיה.

- עובדות בנויות מביטויים. חוקים בנויים מביטוי שנגרר ע"י אוסף של ביטויים אחרים.

בצורה בלתי מפורשת – פרולוג אומר לנו גם איך לבצע את הדברים (ולא רק "מה" לבצע), באמצעות הסדר שהוא משנה מאוד שכתובים הדברים. עם זאת, מתייחסים לפרולוג בצורה הצהרתית שאומרת "מה" לעשות. נשים לב כי כל חוק ניתן להפוך לביטוי לוגי: נוסחה לוגית מסדר ראשון.

- היתרונות של שפות הצהרתיות היא שיש סמנטיקה די בסיסית שקל להוכיח עליה בצורה מתמטית ואוטומטית את הטענות.
- אין side effects שיש בשפות אחרות , אך יש לשים לב לסדר שבו כתובים הדברים.

<u>מה אם היינו מגדירים את האב קדמון קודם הפוך?</u>

```
Ancestor (X,Y) :-
Parent (Z,Y), ancestor(X,Z).
Ancestor (X,Y):- parent(X,Y).
```

אנחנו נקבל שגיאה! נקבל לולאה אינסופית. אם נריץ ancestor(Kim, holly) הוא ינסה הרי קודם למצוא Z והוא הרי לא ימצא ולכן יתקע בלולאה אינסופית. לכן: הסדר חשוב!

- כל שאילתא נענית ע"י התאמה לאטומים – זה נקרא **יוניפיקציה**. אם רוצים למצוא השמה של ערך למשתנה זה נקרא יוניפיקציה.

אנחנו יכולים את העובדות לציין באמצעות מבנים מורכבים יותר. למשל היחס *born* מדרגה 2:

```
Born (person (Alan, Turing), date (23,jun,1912))
```

על מסד נתונים כזה ניתן לבצע למשל שאילתא כמו השאילתה הבאה:

```
?- born (person (ada,Last),data(Day,Month,Year)).
```

מנסה לענות על השאלה – האם יש בן אדם בשם ada עם שם משפחה כלשהו – שנולד בתאריך כלשהו?

<u>סוגים שונים של terms:</u>

<u>אופרטורים:</u>

נתבונן למשל ב – 3*2+1. זהו מבחינת פרולוג לא ערך מספרי אלא ביטוי. ישנם כמה אופרטורים מוגדרים – זהו פרדיקט.

1. אופרטור השוויון =: האופרטור = אומר כי ניתן לבצע יוניפיקציה לשני המשתנים. למשל:

```
?- =(parent(Adam, Seth),parent(adam,X)).

X = seth.
```

האם ניתן למצוא השמה עבור X כך ששני הביטויים הללו בתוך הסוגריים יהיו שווים. נשים לב – זה לא אם התוצאה שלהם היא true, צריך שהביטויים ממש יהיו שווים. הדרך היחידה כאן לקבל שוויון היא כי X=seth. כלומר הדגש כאן הוא שהשוויון מכריע על יונפיקציה למשתנים.

- נשים לב כי:

```
? – X = +(1,*(2,3))

X=1+2*3
```

כלומר הוא מחזיר ביטוי ולא את הערך 7. וכמו כן: אם נשאל אותו האם 7=1+2*3 הוא יחזיר false – זהו לא אותו ביטוי.

2. <u>האופרטור שונה X\=Y:</u> כאן – האם אי אפשר לגרום להם להיות שווים. כלומר, לא ניתן לבצע יוניפיקציה כך שיהיו זהים.
3. <u>האופרטור X==Y:</u> האם הביטויים ממש שווים, לא ע"י השמות ביוניפיקציה אלא ביטויים ממש שווים.
4. <u>האופרטור X==\Y:</u> שוב, הביטויים ממש לא שווים.

Terms	=	\=	==	==\
a,a	Yes	No	Yes	No
a,b	No	Yes	No	Yes
X,Y	Yes (X=Y)	no	No	Yes

דוגמה חשובה: אמרנו - הסדר חשוב. בדוגמה הראשונה קודם בודקים אם X==Y. התשובה שלא, לכן מוחזר false. בדוגמה השנייה, קודם בודקים אם קיימת יוניפיקציה למשתנים, וקיימת X=Y. ואז: הבדיקה X==Y אכן עוברת.

```
? – X ==Y, X=Y.

False.

?-X=Y, X==Y.

X=Y.
```

<u>רשימות:</u>

חלק זה יהיה קצת ארוך, ויימשך בהרצאה הבאה.

יש הבדלים בין הגרסאות של פרולוג, לכן נדבוק בגרסה SWI-Prolog 7+. מדובר במבנ"ת יותר מורכב אך שימושי מאוד.

- סוגריים מרובעים [] מסמל רשימה ריקה.

Term denoted (SWI-Prolog 7+)	List notation
[]	[]
'[|](1,[])'	[1]
'[|](3,[])'(2,'[|]'(1,'[|]'(1,'[|]'(1